ECS two-GPU release scope

This is the settled scope for the ECS (Expert Coordination System) work on two RTX 3080 Ti cards. Everything in "Validated" was measured on that host with the commands shown. Everything in "Not in this release" is either unimplemented, implemented but unverified on hardware, or actively blocked by the transport.

Treat the two-GPU form as the supported configuration until the multi-GPU revision lands.

Hardware

Item	Value
GPU	2x NVIDIA GeForce RTX 3080 Ti, 12 GB, SM 8.6
CPU / RAM	Intel Xeon Silver 4214R, 440 GiB
OS / Python	Ubuntu 22.04, Python 3.12.3
PyTorch / CUDA	2.5.1+cu121 / CUDA 12.1
Model	DeepSeek-V2-Lite-Chat Q4_K_M GGUF (10,364,416,320 bytes)

The two GPUs report no peer-to-peer access, so every cross-device activation crosses PCIe.

Validated

Expert-sharded execution with exact token parity

Rank 0 keeps attention, shared experts and the LM head; rank 1 is a lightweight routed-expert executor that opens only the GGUF expert bytes. The remote worker's peak allocation is ~38-62 MiB because it never loads the transformer.

Single request, `5-2=3`, token ids `[207, 20, 12, 17, 28, 18, 100001]`, identical to the single-GPU baseline at every batch size tested.

Continuous batching

Dynamic admission into free slots, request removal on completion, per-request KV in a paged block pool, prefix block sharing, and an admission grace window.

See `docs/REMOTE_ECS_2X3080TI.md` for the full experiment log.

Workload (dual 3080 Ti)	Result
3B-scale fixed batch 8, aggregate decode	25.90 tok/s (6.17x single GPU)
4 prompts, 44-token, dense slot cache	7.004 s / 3.855 tok/s
4 prompts, 44-token, paged KV	**5.965 s / 4.526 tok/s**
4 requests sharing an 89-token prefix, paged + prefix cache	**6.151 s** vs 8.359 s without reuse (-26%)
per-request prefill with prefix hit	0.55 s vs 1.17 s (-53%)

Every paged run finishes with `used_blocks = 0`: the KV pool is fully returned.

Failure handling

`scripts/run_ecs_expert_parallel_supervised.py` restarts both ranks when the remote rank dies (verified with an injected fault, `DEEPINFER_REMOTE_FAULT_AFTER`), the remote worker writes a heartbeat the coordinator monitors, and the NCCL process group now has a timeout so a dead rank raises instead of hanging.

Not in this release

Multi-GPU worker pooling - implemented, unverified

`PooledExpertClient` exists (`deepinfer/moe/expert_worker_pool.py`): least-in-flight dispatch, reply ownership, aggregated metrics. It is covered by unit tests with fake transports and **has never carried real NCCL traffic**, because pooling needs a coordinator plus two expert workers and this host has two GPUs.

Prefill/decode overlap - blocked

Overlapped admission aborts with `SIGABRT` on the NCCL path even after submissions were funnelled through a single I/O thread. A `ProcessGroupNCCL` runs point-to-point work on one internal communication stream, so two forwards cannot share it. `--overlap-prefill` stays experimental and off by default.

Collective all-to-all

The in-model activation exchange is point-to-point request/reply. A separate ExpertPacket all-to-all smoke passed, but the runtime does not use it.

In-flight resume

A crashed worker costs the whole request. The supervisor retries from scratch; KV state is not recovered.

What that costs

The honest summary is that the two-GPU form is a **working, reproducible, token-exact** distributed MoE runtime, not a throughput machine. The single remote worker serialises every expert chunk, so adding requests past the point where the worker saturates buys latency, not throughput. It is useful for running a model that does not fit on one card, and for the engineering record; it is not competitive with llama.cpp on raw tokens per second.

Upgrade path to multi-GPU

1. Replicate and supervise workers (needs at least three ranks).
2. Shard activations by expert ownership with a per-worker route table instead of sending whole chunks to one worker.
3. Replace point-to-point request/reply with a collective all-to-all, which is also the prerequisite for prefill/decode overlap.
4. Add in-flight request/KV resume so a worker restart does not discard work.

Reproduction

```
# single GPU correctness baseline
PYTHONPATH=$PWD python scripts/smoke_deepseek_v2_lite_moe.py \
  --prompt "只回答数字：5-2=?" --max-new-tokens 8

# dual GPU continuous batching, paged KV
PYTHONPATH=$PWD torchrun --standalone --nproc_per_node=2 \
  scripts/bench_ecs_paged_continuous_batch.py \
  --max-batch-size 4 --max-new-tokens 8 --arrival-delay 0.15 --max-cache 256

# dual GPU continuous batching with prefix sharing
PYTHONPATH=$PWD torchrun --standalone --nproc_per_node=2 \
  scripts/bench_ecs_paged_continuous_batch.py \
  --max-batch-size 4 --max-new-tokens 8 --arrival-delay 0.15 --max-cache 256 \
  --prefill-mode paged --prefix-cache-blocks 64

# supervised restart
PYTHONPATH=$PWD python scripts/run_ecs_expert_parallel_supervised.py
```

Result JSONs live in `docs/results/`; the full experiment log, including the failed ones, is `docs/REMOTE_ECS_2X3080TI.md`.